

GENERATIVE AI

From Transformers to AI Agents

NATIONAL INSTITUTE OF ELECTRONICS AND INFORMATION TECHNOLOGY, ROPAR

NIELIT Ropar · AI with ML · 2025–26

Instructor: Project Engineer, Department of Training

What We'll Cover

01

Intro to Gen AI

Concepts & real-world examples

02

Foundation Concepts

Tokens, embeddings, context

03

Transformers

Attention & architecture

04

LLMs

GPT, Llama, Claude & more

05

Prompt Engineering

Zero-shot to CoT

06

RAG

Retrieval-augmented pipelines

07

Fine-Tuning

LoRA, QLoRA, PEFT

08

AI Agents

Tools, memory, planning

09

Multimodal AI

Vision-language models

10

Industry & Ethics

Applications & responsibility

11

Projects

Hands-on builds

12

Career Roadmap

Skills & path forward

SECTION 01

Introduction to Generative AI

What, Why & How it changed everything

What is Generative AI?

TRADITIONAL AI

- Classifies / predicts from existing data
- Pattern recognition & decision-making
- Examples: spam filter, image classifier
- Output: label, score, or class

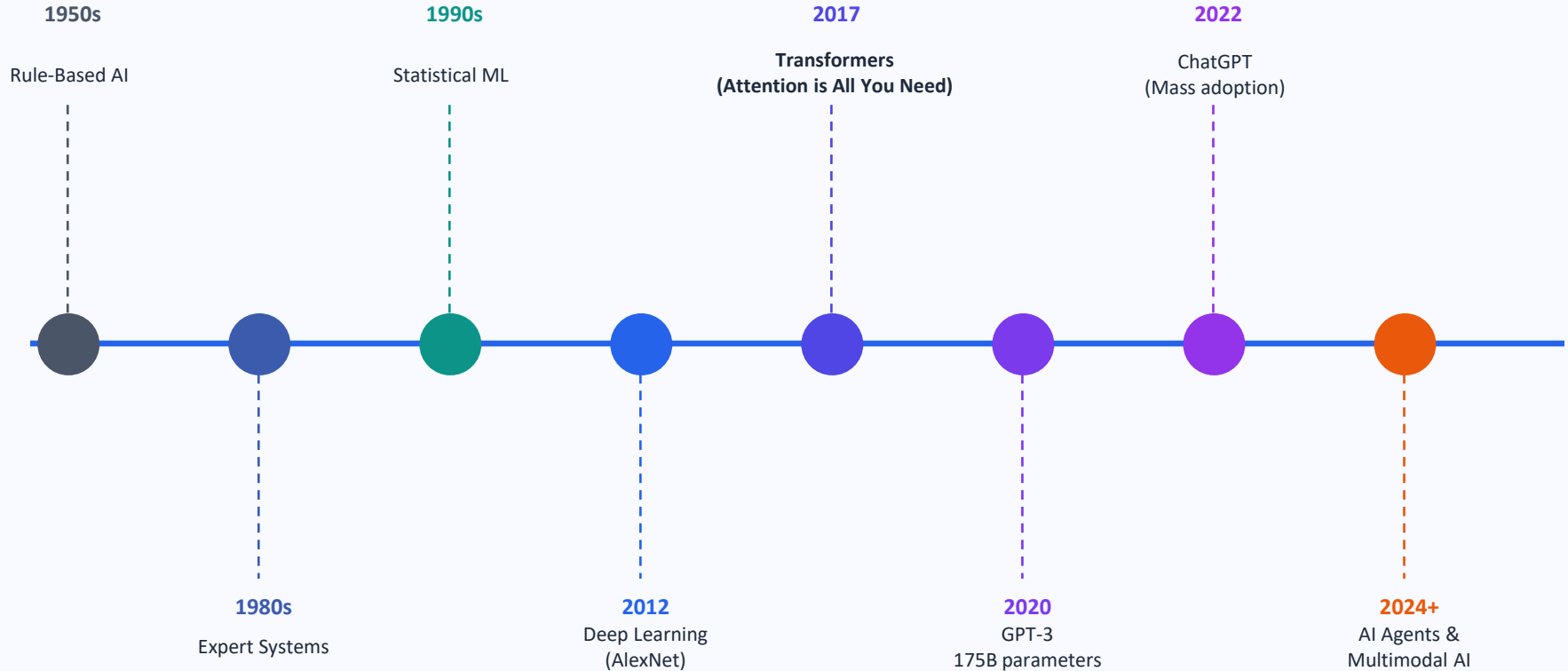
vs

GENERATIVE AI

- Creates NEW content from learned patterns
- Text, images, audio, code, video
- Examples: ChatGPT, DALL-E, GitHub Copilot
- Output: novel, human-like content

Note: Generative AI doesn't just analyze — it creates. This distinction is what makes it revolutionary for engineering, media, healthcare and more.

Evolution of AI: A Timeline



Where Generative AI is Used Today



Conversational AI

ChatGPT, Gemini, **Claude**, Sarvam.ai



Image Generation

Sora, Nano Banana, Midjourney, DALL-E, Stable Diffusion



Code Assistants

Antigravity, GitHub Copilot, Cursor, Tabnine



Audio / Music

Gemini, ElevenLabs, Suno AI, Whisper



Video Generation

Runway ML, Sora, Pika Labs, Seedance 2.0



Document AI

NotebookLM, PDF chatbots, Summarizers

SECTION 02

Foundation Concepts

Tokens · Embeddings · Vector Representations

Tokenization: How AI Reads Text

AI doesn't read words — it reads TOKENS (chunks of text)

Input: "The quick brown fox jumps over the lazy dog"



Key Facts:

- 1 token $\approx$ 4 characters or $\frac{3}{4}$ of a word
- GPT-4 has a 128,000 token context window
- Tokenization affects cost — more tokens = more cost
- Common tokenizers: BPE (Byte Pair Encoding), WordPiece, SentencePiece

Token Count	
Text	Tokens
Hello World	2
The lazy dog	3
Supercalifragilistic	6
Code: print()	4

Embeddings: Meaning as Numbers

Words are converted to high-dimensional vectors that capture semantic meaning

King	[0.91, 0.23, 0.78...]
Queen	[0.89, 0.21, 0.80...]
Dog	[0.12, 0.88, 0.02...]
Cat	[0.11, 0.85, 0.04...]

Key Insight

King – Man + Woman
 $\approx$ Queen

Similar meaning
= similar vectors

Semantic search uses
cosine similarity
between vectors

SECTION 03

Transformers

The architecture that changed everything

Why RNNs & LSTMs Weren't Enough

Vanishing Gradients

Long sequences lose early context during training. Gradients shrink to near zero, making learning impossible.

Sequential Processing

Cannot parallelize — each word waits for the previous. Makes training on large datasets extremely slow.

Limited Long-Range Dependency

Struggle to connect distant words. 'The cat that sat on the mat was ____' — hard to recall 'cat' at the end.

Fixed Context Window

Memory cell has fixed capacity. Cannot efficiently scale to thousands of tokens.

Attention: The Core Idea

"Attention is ALL You Need" — Vaswani et al., Google Brain (2017)

The research paper that created Trillion Dollars Industries

How Attention Works:

- 1 Every word looks at ALL other words
- 2 Compute similarity scores (Query $\times$ Key)
- 3 Normalize scores using Softmax
- 4 Weighted sum of Value vectors = output

Key Formula: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$

Example Sentence

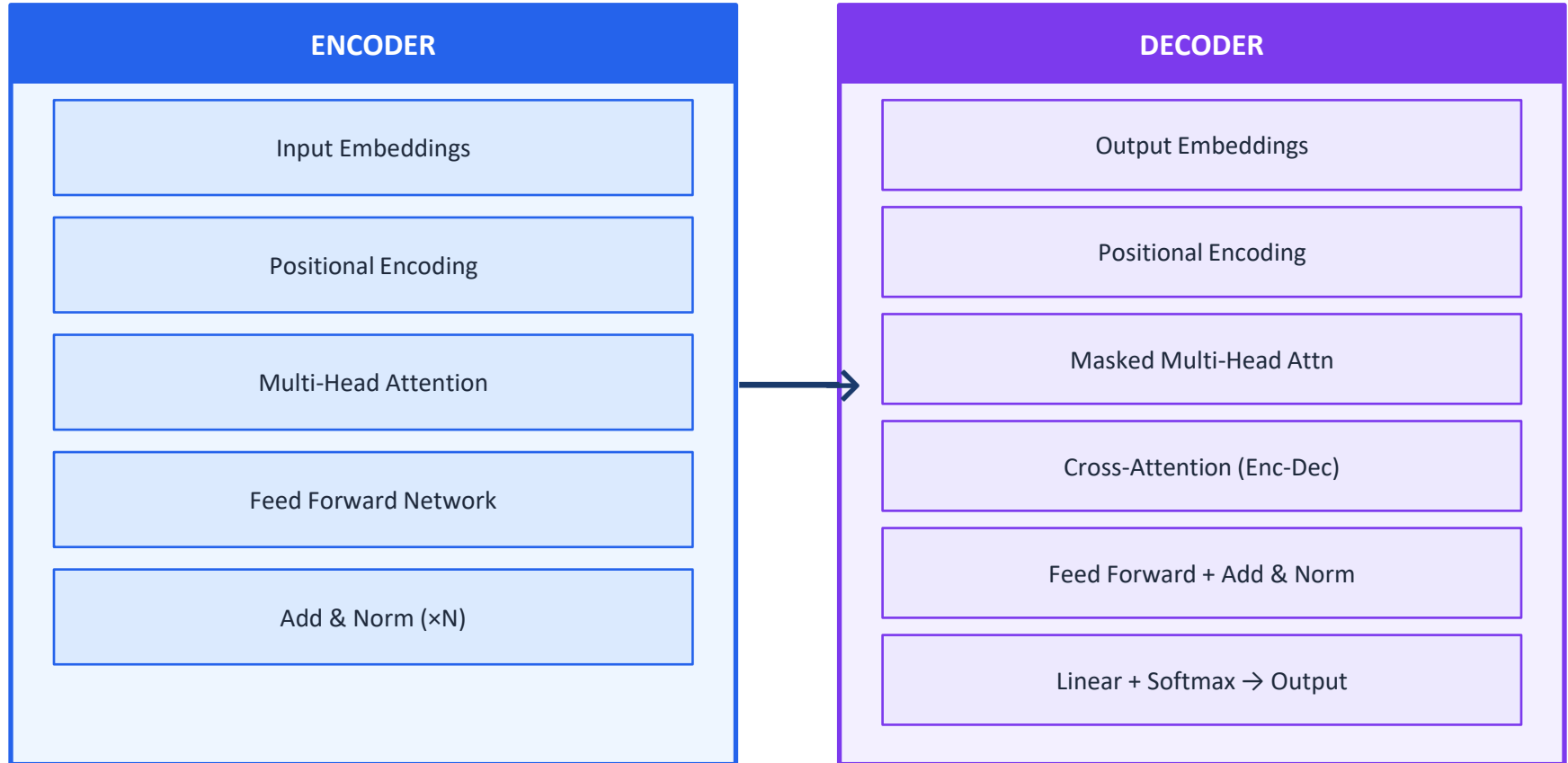
"The bank by the river was steep"

For "bank" $\rightarrow$ high attention to "river" (bank = riverbank)

"The bank froze my account"

For "bank" $\rightarrow$ high attention to "account" (bank = financial)

Transformer Architecture





SECTION 04

Large Language Models (LLMs)

GPT · Llama · Gemma · Claude



Large Language Models (LLMs)

GPT-4

OpenAI
~1.8T (MoE)

Llama 3

Meta
8B–70B

Gemma 2

Google
2B–27B

Claude 3.5

Anthropic
~Unknown

Mistral

Mistral AI
7B–8×22B

How LLMs are Trained:

1

Collect massive text data (web, books, code)

2

Tokenize & create training batches

3

Pretraining: predict next token (self-supervised)

4

RLHF: align with human preferences

5

Fine-tune for specific tasks

6

Deploy for inference

LLM Hallucinations & Decoding Parameters

⚠ Hallucinations

What: LLMs confidently generate false info

Why: Trained to predict plausible text, not verify facts

Examples:

- Fabricating citations & research papers
- Inventing historical facts
- Hallucinating code that looks right but breaks

Fix: RAG, fact-checking, lower temperature

🧠 Decoding Parameters

Temperature:

Controls randomness. Low = focused, High = creative

Top-p (Nucleus):

Sample from top-p probability mass

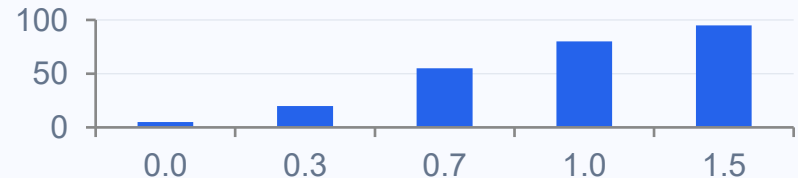
Top-k:

Sample from top-k most likely tokens

Max Tokens:

Limit the output length

Temperature Effect on Output Diversity



SECTION 05

Prompt Engineering

Zero-shot · Few-shot · Chain-of-Thought · Role prompting



Prompting Techniques

Zero-Shot

Prompt:

Classify: "I love this product!" →

→ Positive

Few-Shot

Prompt:

Positive: "Great!"

Negative: "Terrible!"

Classify: "Okay..." →

→ Neutral

Chain-of-Thought

Prompt:

Q: $23 \times 4 = ?$

A: 23×4 : $20 \times 4 = 80$, $3 \times 4 = 12$, total = 92 →

→ 92

Role Prompting

Prompt:

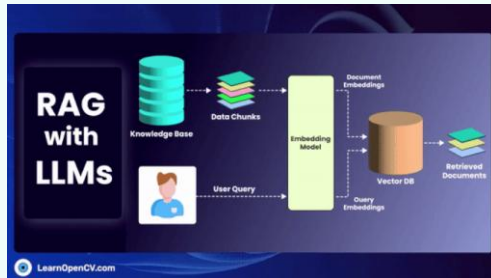
"You are a senior Python developer. Review this code:"

→ Detailed code review...

SECTION 06

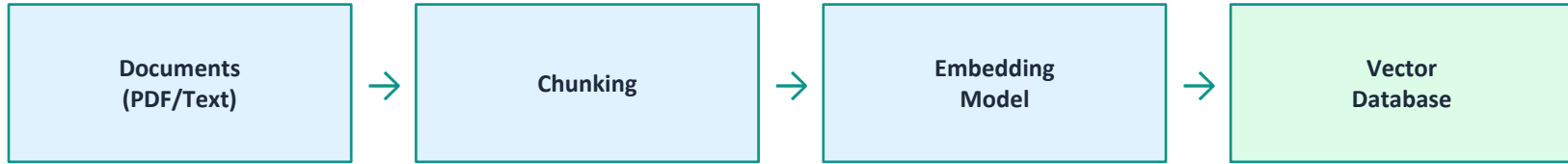
Retrieval-Augmented Generation (RAG)

Grounding LLMs with real-world knowledge

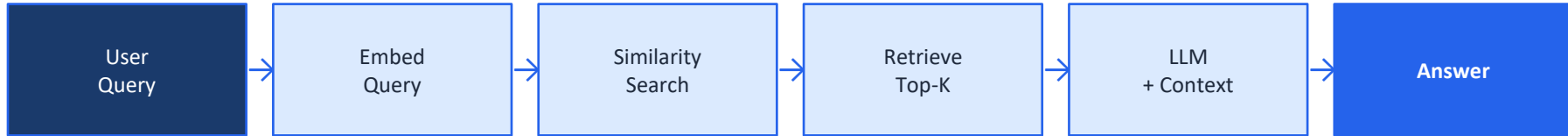


RAG Pipeline Architecture

OFFLINE: Document Indexing



ONLINE: Query & Generation



Why RAG?

- ✓ No hallucinations on known facts
- ✓ Uses up-to-date documents
- ✓ Cites sources
- ✓ No expensive fine-tuning needed
- ✓ Domain-specific knowledge

Tools: LangChain · LlamaIndex · FAISS · ChromaDB · Pinecone · Weaviate

SECTION 07

Fine-Tuning LLMs

LoRA · QLoRA · Unsloth · PEFT · Hugging Face

Fine-Tuning Approaches

Full Fine-Tuning

Update ALL model weights
Requires huge compute
Best performance
Risk: catastrophic forgetting

✓ Best accuracy ⚠ Very expensive

LoRA (Low-Rank Adaptation)

Inject small trainable matrices
Freeze original weights
<<1% of parameters trained
Runs on consumer GPUs

✓ Efficient ⚠ Slight quality gap

QLoRA

LoRA + 4-bit quantization
Further reduces memory
Train 70B models on 1 GPU
Best efficiency/quality ratio

✓ Very memory efficient ⚠ Slower training

PEFT (General)

Parameter-Efficient Fine-Tuning
Umbrella for LoRA, Prefix-Tuning, Adapters
Hugging Face PEFT library
Task-specific adaptors

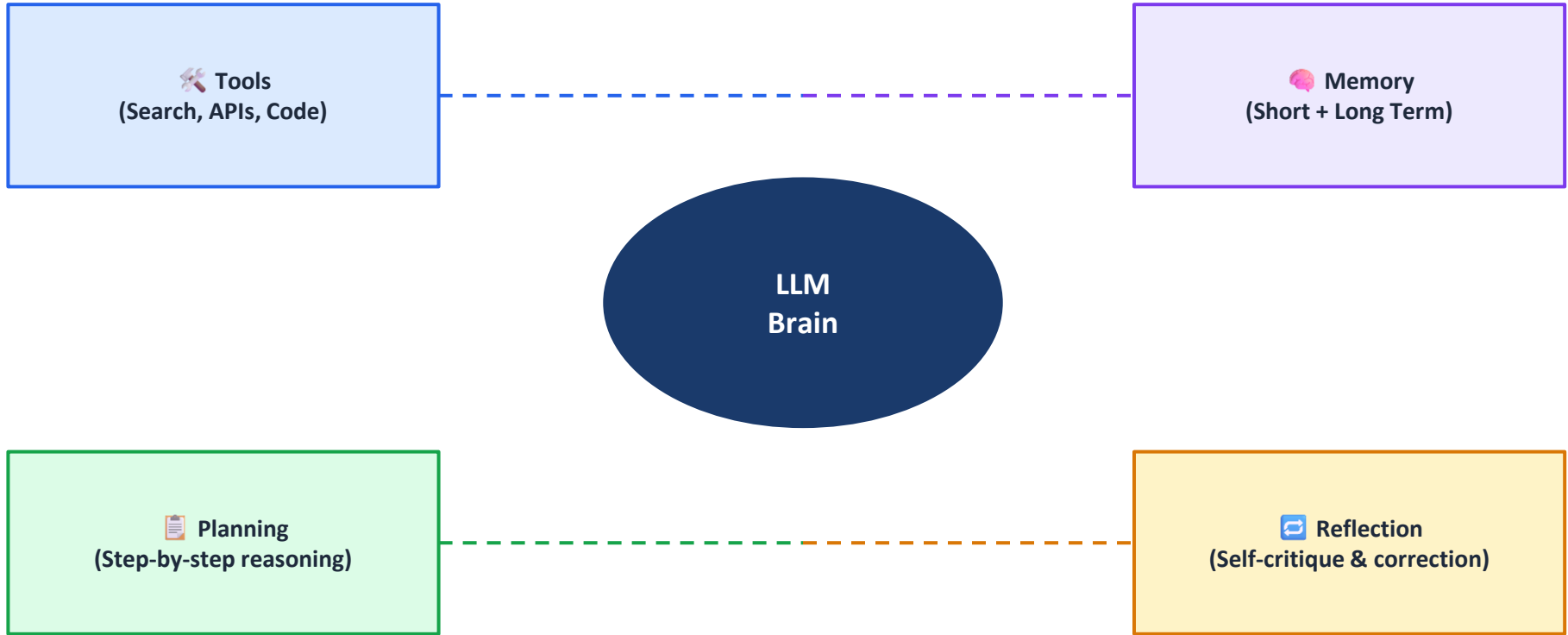
✓ Modular ⚠ Added complexity

SECTION 08

AI Agents (Agentic AI)

Tools · Memory · Planning · Autonomous Workflows

What is an AI Agent?



ReAct Loop: Observation → Thought → Action → Observation → Thought → Action → Final Answer

Frameworks: CrewAI · LangGraph · AutoGen · OpenAI Swarm

Multi-Agent Systems



Example: "Write a blog on AI trends" → Research Agent gathers data → Writer drafts → Critic reviews → Coder adds code examples

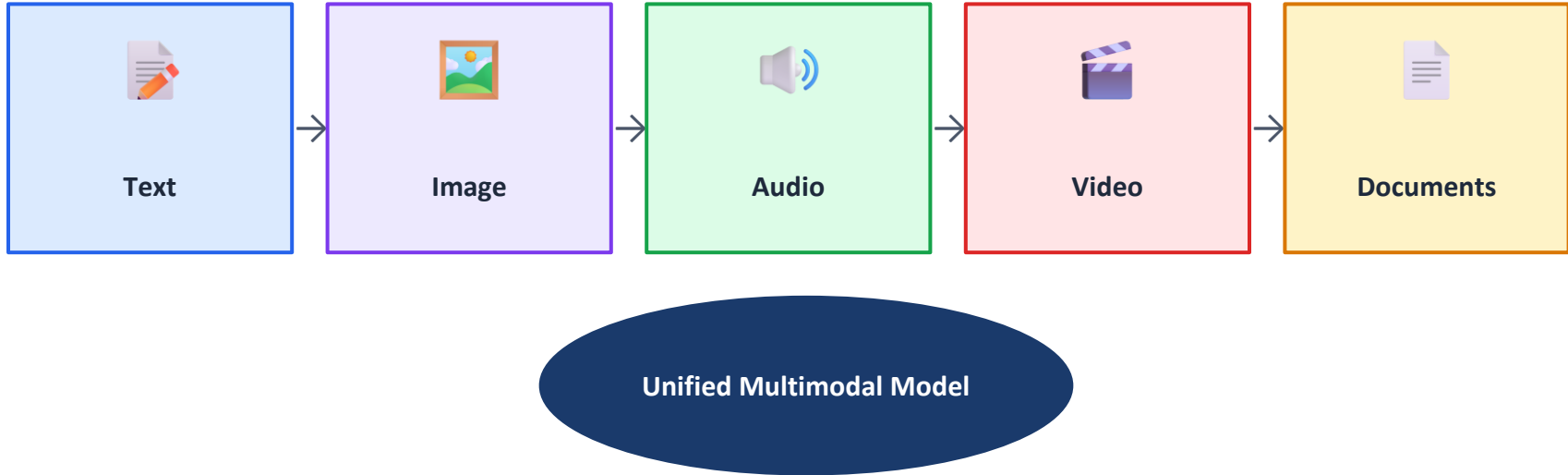
Try: <https://www.kimi.com/agent>

SECTION 09

Multimodal AI

Text + Image + Audio + Video

Multimodal AI: Beyond Text



Key Multimodal Models:

	GPT-4o	OpenAI — Text, Image, Audio, Video		Gemini 1.5	Google — Text, Image, Audio, Video, 1M context
	LLaVA	Open-source Vision-Language Model		Florence-2	Microsoft — Image understanding & captioning
	Claude 3.5 Sonnet	Anthropic — Vision + Text reasoning			

SECTION 10

Industry Applications & Ethics

Real-world impact and responsible AI

Generative AI Across Industries



Healthcare

Medical report generation, drug discovery, diagnostic imaging AI, patient chatbots



Education

Personalized tutors, quiz generation, grading automation, adaptive content



Cybersecurity

Threat detection, log analysis, phishing email detection, automated incident reports



Legal AI

Contract analysis, case research, summarization, compliance checking



Software Dev

Code generation, debugging, documentation, testing, refactoring at scale



Manufacturing

Defect detection, maintenance prediction, automated quality reports

Risks & Responsible AI



Hallucinations

AI generates confident but false information

High



Deepfakes

Synthetic media used for misinformation & fraud

Critical



Bias & Fairness

Training data bias leads to discriminatory outputs

High



Privacy

Models may memorize and leak sensitive training data

Medium



Job Displacement

Automation risk for repetitive cognitive tasks

Medium



AI Safety

Misalignment between AI goals and human values

Critical

SECTION 11

Hands-On Projects

Build real AI applications from scratch

Mini-Project Ideas

Beginner · AI Chatbot

Stack: Python + LangChain + Ollama/OpenAI

Build a conversational chatbot with memory and conversation history

Beginner · PDF Q&A Bot

Stack: LangChain + FAISS + Streamlit

Upload any PDF and ask questions — RAG pipeline implementation

Intermediate · AI Voice Assistant

Stack: Whisper + LLM + TTS

Speech-to-text → LLM reasoning → text-to-speech response system

Intermediate · Custom LLM Fine-Tune

Stack: Hugging Face + LoRA + Colab

Fine-tune Llama/Mistral on a domain-specific dataset using QLoRA

Advanced · Multi-Agent Research Tool

Stack: CrewAI + LangGraph + Tools

Multi-agent system that researches, writes, critiques & publishes reports

Advanced · Multimodal Medical AI

Stack: LLaVA + Gradio + FastAPI

Image + text input for medical report generation and diagnostic assistance

Code Snippet: Simple RAG Pipeline

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings, ChatOpenAI

# 1. Load PDF
loader = PyPDFLoader("document.pdf")
chunks = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50
).split_documents(loader.load())

# 2. Embed & Store
db = FAISS.from_documents(chunks, OpenAIEmbeddings())

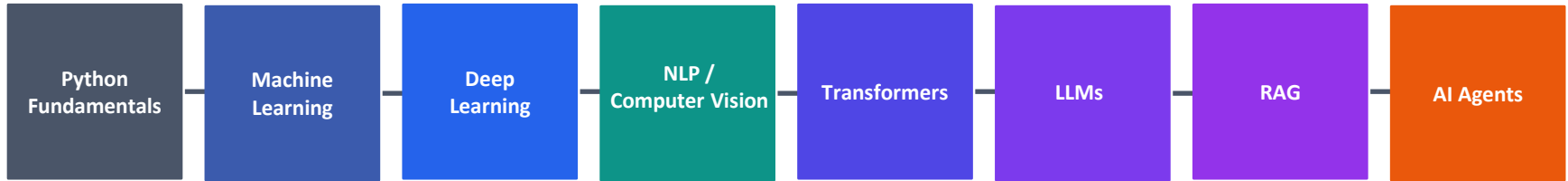
# 3. Query
results = db.similarity_search("What is RAG?", k=3)
```

SECTION 12

Career Roadmap

Your path from student to AI Engineer

AI/ML Engineer Roadmap



Tools to Master:

Languages:

Python, JavaScript

ML/DL:

PyTorch, **TensorFlow**,
Keras

LLM Frameworks:

LangChain, LlamaIndex,
HF Transformers

Vector DBs:

FAISS, ChromaDB,
Pinecone

Cloud & MLOps:

AWS/GCP/Azure, Docker,
MLflow

Agent

Frameworks:

CrewAI, LangGraph,
AutoGen

Portfolio & Interview Preparation

Build Your Portfolio

- ✓ GitHub with 10+ AI/ML projects
- ✓ Hugging Face profile with models/spaces
- ✓ Kaggle competitions (bronze or above)
- ✓ Technical blog on Medium/Substack
- ✓ LinkedIn with project posts
- ✓ Demo videos on YouTube
- ✓ Personal portfolio website

Interview Topics

- Q: Explain Attention mechanism
- Q: Difference: LoRA vs Full fine-tuning
- Q: Design a RAG system
- Q: Handle LLM hallucinations
- Q: ML system design questions
- Q: Python: implement tokenizer
- Q: Explain Transformer from scratch

Key Takeaways

01

Transformers (2017) revolutionized NLP by replacing RNNs with self-attention, enabling massively parallel training and long-range context understanding.

02

LLMs are trained on massive corpora to predict the next token, emerging powerful capabilities like reasoning, coding, and multi-turn dialogue.

03

Prompt Engineering is a skill — zero-shot, few-shot, and chain-of-thought techniques dramatically improve LLM outputs without any training.

04

RAG combines retrieval with generation to ground LLMs in real, up-to-date facts and eliminate hallucinations for domain-specific applications.

05

AI Agents extend LLMs with tools, memory, and planning — enabling autonomous, multi-step task execution across domains.

06

Responsible AI is not optional — bias, privacy, deepfakes, and safety risks must be addressed at every stage of AI development.

The Future of Generative AI



AI + Science

Drug discovery, protein folding, climate modeling



Embodied Agents

Robots that perceive, plan, and act in the physical world



Reasoning Models

o1, o3 — models that 'think before they answer'



Personal AI

On-device models customized to each individual



AI-Native Apps

Applications built around LLMs at their core



AI Regulation

EU AI Act, governance frameworks worldwide



Resources & Further Learning

Papers

- Attention is All You Need (2017)
- GPT-3: Language Models are Few-Shot Learners
- LoRA: Low-Rank Adaptation (2021)
- RAG: Retrieval-Augmented Generation (2020)
- ReAct: Synergizing Reasoning and Acting (2022)

Courses

- fast.ai — Practical Deep Learning
- DeepLearning.AI — LLM Specialization
- Hugging Face — NLP Course (free)
- Stanford CS224N — NLP with DL
- NPTEL — Deep Learning (IIT)

Tools

- Hugging Face (huggingface.co)
- LangChain Docs (langchain.com)
- Ollama — run local LLMs
- Gradio + Streamlit for demos
- Google Colab for free GPU

Q & A

Questions & Discussion

Ask anything about Generative AI, Transformers, LLMs, RAG, or AI Agents

lovnishverma.in · @lovnishverma · NIELIT Ropar

Thank You!

Generative AI: From Transformers to AI Agents



Portfolio:

lovnishverma.in



GitHub:

github.com/lovnishverma



Hugging Face:

huggingface.co/lovnishverma



Email:

princelv84@gmail.com



NIELIT Ropar:

Project Engineer — AI & ML, Big Data and Data Science